Internet of Things 33 (2025) 101705

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot

Research article

Fast and low cost FPGA-based architecture for arrhythmia detection with CNN

Luca Greco, Francesco Moscato *, Pierluigi Ritrovato, Mario Vento

Department of Information and Electrical Engineering and Applied Mathematics, University of Salerno, Invariante 12/B, Via Giovanni Paolo II, 132, 84084 Fisciano (SA), Italy

ARTICLE INFO

Keywords:
CNN
FPGA
Machine learning

ABSTRACT

Deep Neural Networks have been applied in many fields and have exhibited extraordinary abilities. However, many challenges arise when dealing with embedded or low-resource computing architectures in different contexts like healthcare or IoT in Industry 4.0. In recent years, rapid growth has been seen in using machine learning techniques to interpret sensor data in healthcare applications. Convolutional Neural Networks (CNNs) are highly effective, but they have a significant drawback: they require large amounts of computational resources, usually available only "on the Cloud". Edge and Fog nodes in healthcare applications (e.g. wearable sensors) are generally ill-suited to running CNN models with requirements like low computational resources, real-time execution, (very) low power consumption or low intrusiveness. In order to get through these difficulties, we propose a solution based on novel data-flow architectures and layer partitioning that enables fast classification in CNNs even when dealing with low resources. We apply our approach in developing a classifier (based on CNNs) for arrhythmia detection, which maintains good precision on low-power and low-cost FPGAs. We prove that the presented approach is general enough to distribute computation on parallel FPGAs too. Results show interesting performance improvements even when using low-resource hardware to implement the classifier.

1. Introduction

The Internet of Medical Things (IoMT) encompasses a wide range of interconnected medical devices, sensors, and systems that collect and exchange data to facilitate healthcare services [1]. These devices generate a massive amount of data, requiring efficient processing [2] and analysis to derive meaningful insights. The emergence of IoMT has paved the way for advanced healthcare solutions that improve patient care, enable remote monitoring, and enhance medical diagnostics [3–5]. In particular, cardiovascular diseases are a significant cause of morbidity and mortality worldwide. One crucial aspect of diagnosing cardiovascular diseases is the accurate detection and classification of arrhythmia, which are abnormal heart rhythms. Electrocardiogram (ECG) signals provide valuable information for arrhythmia detection and several approaches have been proposed for automatic ECG classification through the years.

Recently, it has been shown that traditional machine learning methods have less arrhythmia detection accuracy than neural network-based approaches on several databases: best performing SVM-based classifiers [6] have an accuracy of 87% whereas neural network-based methods (especially Convolutional Neural Networks - CNNs) can reach up to 98.42% [7] or 99.01% [8] accuracy in test phase. CNN-based methods do not require computational overhead for separate feature extraction and classification since they

* Corresponding author.
E-mail addresses: lgreco@unisa.it (L. Greco), fmoscato@unisa.it (F. Moscato), pritrova@unisa.it (P. Ritrovato), mvento@unisa.it (M. Vento).

https://doi.org/10.1016/j.iot.2025.101705
Received 14 April 2025; Received in revised form 28 June 2025; Accepted 7 July 2025
Available online 17 July 2025
2542-6605/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

learn high-level features from input and exhibit high classification and prediction capabilities on publicly available datasets [9]. However, the computational demands of neural networks require efficient implementations to enable real-time processing and portability. Field-Programmable Gate Arrays (FPGAs) offer a viable solution for accelerating neural network computations, making them well-suited for physiological signal analysis, more in particular for ECG arrhythmia classification.

FPGAs are reconfigurable integrated circuits that can be programmed to perform customized computations efficiently. They provide high parallelism and low power consumption, making them attractive for implementing neural network architectures. FPGA-based implementations of CNNs have gained attention due to their ability to capture spatial dependencies in ECG signals effectively. By leveraging the parallelism of FPGAs, CNNs can process ECG signals in real-time, facilitating rapid and accurate arrhythmia classification.

The primary objective of the research community in this field is to develop FPGA-based neural network implementations for the analysis of signals with high accuracy, low latency, and low power consumption. The FPGA implementations aim to overcome the computational challenges posed by real-time ECG analysis, enabling faster and more efficient diagnosis of arrhythmia. Moreover, FPGA-based implementations provide the potential for portable and wearable ECG monitoring devices, especially in the context of the Internet of Medical Things (IoMT), empowering continuous monitoring and timely intervention for individuals affected by cardiovascular diseases.

In recent years, several studies have focused on developing FPGA-based CNN implementations for ECG arrhythmia detection [10–12]. These studies have explored various aspects, including optimization techniques, memory management strategies, and model architectures, to enhance the performance and efficiency of the FPGA implementations. This is the reason this work addresses CNNs, proposing a novel FPGA-based neural network implementation for detecting arrhythmia using ECG signals. The main goal is to provide fast responses for CNN execution even on very cheap hardware, with low power consumptions. This favors smaller, less expensive and less invasive systems in the healthcare sector, with the aim of keeping the performance, accuracy and reliability of the proposed system sufficiently close to those of the most performing but more expensive and power-consuming cloud-based systems.

The architecture we introduce is inherently scalable and adaptable across various FPGA platforms. It enables designers to balance trade-offs between spatial complexity, power consumption, and computational performance, depending on application-specific requirements.

The core principles of this design include: (a) the duplication of computational units that form the CNN layers, both temporally (through iterative reuse) and spatially (via parallel instantiation); (b) the optimized management of memory resources for storing CNN weights as well as the inputs and outputs of convolutional layers; (c) a bus architecture that efficiently handles the *routing* of intermediate results between successive layers, ensuring data is available when needed.

This system is intended to leverage CNNs that are pre-trained using conventional high-performance computing resources. To simplify hardware implementation, the architecture employs fixed-point arithmetic and quantization techniques. Although these methods can compromise model accuracy, our design and methodology aim to mitigate the typical losses in performance and precision that often accompany such optimizations.

The remainder of this paper is organized as follows. Section 2 provides background and reviews key research works, briefly discussing their contributions, methodologies, and achieved results. The insights gained from these studies have shed light on the potential of FPGA-based implementations in revolutionizing the field of ECG arrhythmia detection, leading to faster and more accurate diagnoses, improved patient care, and enhanced monitoring capabilities. Section 3 discusses in detail the architecture of the proposed FPGA accelerator. Section 4 illustrates the experimental setup, while in Section 5, achieved results and a comparison with recent state-of-the-art methods is performed. Finally, Section 7 draws the conclusions.

2. Background and related work

2.1. Domain overview and key concepts

Early and reliable detection of arrhythmias from electrocardiogram (ECG) signals plays a vital role in the prompt diagnosis and management of cardiovascular conditions. Traditional detection methods often rely on manually designed features and deterministic rule-based algorithms, which may struggle to capture the complex temporal and morphological patterns inherent in ECG data.

Convolutional Neural Networks (CNNs) have emerged as an effective paradigm for automatic feature extraction and learning, demonstrating outstanding performance across a wide array of applications in image and signal processing. Architecturally, CNNs are a subset of multilayer feed-forward neural networks characterized by three principal layer types [13]: *convolutional layers* apply trainable kernels to the input data to generate feature maps, extracting localized patterns across the spatial or temporal domain; *pooling layers* reduce the spatial resolution of feature maps while preserving the most salient information, typically through max or average pooling; *fully connected layers* flatten the output of preceding layers into a one-dimensional vector, facilitating classification through probability outputs corresponding to target classes.

Due to their capability to model structured, grid-like data, CNNs are extensively utilized in fields such as computer vision [14], supporting tasks like image classification, object detection, and segmentation [15]. In biomedical contexts, CNNs have shown particular promise for analyzing ECG signals [16], benefiting from their scalability and capacity to generalize from large datasets. Additionally, their relatively simple training process and robust generalization make them suitable for deployment in embedded and real-time systems, such as autonomous vehicles [17].

The implementation of CNNs on Field Programmable Gate Arrays (FPGAs) enables high-throughput, low-latency inference suitable for real-time ECG analysis. This process involves mapping network operations onto FPGA resources, requiring careful architectural and memory optimization. Techniques such as depthwise separable convolutions have been adopted to minimize computational complexity and parameter count without sacrificing model accuracy [18]. Furthermore, strategies like on-chip buffer reuse and optimized memory hierarchies help alleviate bandwidth constraints and improve system performance [19].

Recent works have addressed the challenges of deploying CNNs on FPGAs by introducing design space exploration frameworks that balance accuracy, resource utilization, and energy consumption [20]. Complementary optimization techniques — such as quantization-aware training and structured pruning — have demonstrated success in compressing models while preserving predictive performance [21]. Architectural innovations like systolic arrays have also been explored to accelerate convolutional operations and reduce latency [22,23].

Performance evaluations of FPGA-based CNN accelerators for arrhythmia detection have shown notable gains in speed and accuracy. For instance, works such as [23–25] report classification accuracies exceeding 97%, with ECG sample processing times as low as 0.63 ms and 0.5 ms.

Given the constrained resources of embedded platforms — including microcontrollers (MCUs), FPGAs, and ASICs — there is a growing need to reduce the computational burden of CNNs. This necessity stems from the large parameter count and redundancy common in deep models [26]. As a result, numerous model compression techniques have been developed, including pruning, quantization, and more advanced approaches such as low-rank factorization and knowledge distillation [27].

Pruning involves the removal of non-essential parameters to streamline the model and reduce inference costs [26]. It may be applied statically during training or dynamically at runtime [28], with techniques such as: *weight pruning*, which eliminates individual weights below a certain magnitude threshold [29]; *neuron pruning*, which removes entire neurons along with their associated connections [30]; *filter pruning*, which discards convolutional filters based on ranking metrics like the L1 or L2 norm [31]; *layer pruning*, which involves eliminating less critical layers from the network [31].

Quantization further compresses models by reducing the precision of weights and activations [26]. By representing parameters with lower bit-widths — such as 16-bit, 8-bit, or even binary formats — quantization reduces memory usage and computational load. This allows CNNs to execute more efficiently on hardware with limited resources, while maintaining acceptable accuracy levels [27].

2.2. Existing approaches and prior work

Recent advances in ECG signal processing have highlighted the shift from conventional statistical techniques to data-driven, AI-based methods. Ardeti et al. [32] provide a thorough survey of this transition, discussing how artificial intelligence methods are increasingly employed to enhance signal de-noising, feature extraction, and classification performance in health monitoring systems. Their review underscores that machine learning (ML) approaches consistently outperform classical statistical models in predicting cardiovascular conditions.

Several works have explored FPGA-based implementations to accelerate deep learning models for ECG classification. Aruna et al. [10] introduce a deep convolutional neural network (DCNN) architecture tailored for FPGA deployment, applied to the MIT-BIH Arrhythmia dataset [33]. The proposed system incorporates ENLMS pre-processing and discrete wavelet transform (DWT) for feature extraction, achieving a classification accuracy of 98.6%. Notably, it operates at 185.426 MHz, with a throughput of 35.66 GOPS and a low power footprint of 0.45 mW.

Chen et al. [11] present *AutoMLP*, a framework that automates the design and optimization of multilayer perceptron (MLP) models for FPGA acceleration. The system features an automatic toolchain for selecting optimal bit-widths and data representations. When evaluated on the MIT-BIH atrial fibrillation dataset, a five-layer MLP achieved 94.20% accuracy, consuming 15 mW at a 100 MHz clock frequency.

Hu et al. [34] propose a lightweight classification model based on a dynamically biased Long Short-Term Memory (DB-LSTM) architecture. With just four layers, the model delivers 96.74% accuracy on the MIT-BIH dataset. Even when quantized from FP32 to INT4, the model experiences only a 2.4% reduction in accuracy. Implemented on a Xilinx Artix-7 FPGA, it exhibits a remarkably low dynamic power consumption of 40 μW.

The work by Inadagbo et al. [12] explores a customized Tensor Compute Unit (TCU) generated via the Tensil AI open-source toolchain for deployment on a PYNQ Z1 board. Their FPGA implementation of a ResNet-20 model, trained on MIT-BIH, achieves up to 99% classification accuracy with a total power consumption of 1.53 W.

Ku et al. [35] propose a resource-efficient 1D CNN architecture incorporating quantization and pruning. Their design employs a parallel shift processing element array and is implemented on a Xilinx PYNQ-Z2 board. The system achieves 96.6% accuracy on MIT-BIH arrhythmia data while operating at 10 MHz and consuming 0.131 W of power.

Zhang et al. [36] present a configurable 1D CNN architecture designed for multi-class ECG classification. The system utilizes parallelizable systolic array structures capable of accommodating varying filter sizes and channel configurations. Deployed on a Xilinx Zynq-7000 SoC (XC7Z045) using the ZC706 evaluation board, the design achieves a peak accuracy of 99.3% at 200 MHz with 0.58 W power consumption.

Complementing these efforts, Rawal et al. [9] propose a novel multiplier-less hardware design for 1D-CNN, referred to as Hardware CNN Architecture (HCA). While the overall arrhythmia classification accuracy is 86.37%, the architecture demonstrates up to 97.34% accuracy specifically for atrial fibrillation. The implementation, running on a ZYNQ Ultrascale (ZCU106) FPGA at 100 MHz, exhibits a power usage of 628 mW.

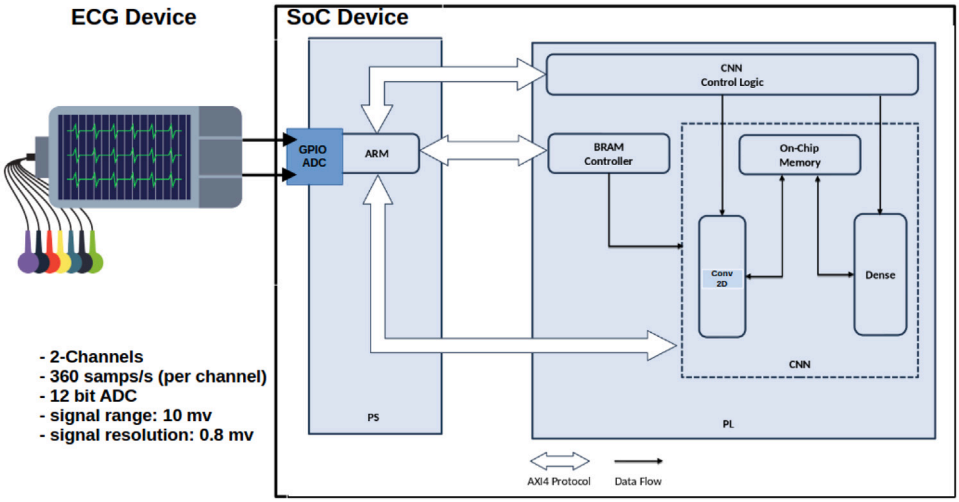


Fig. 1. Overall system architecture.

2.3. Limitations and research gaps

Recent works on FPGA-based CNN accelerators [37–40] typically involve synthesizing the entire CNN architecture directly onto the FPGA fabric. While this method can deliver high throughput, it often necessitates the use of high-end FPGA devices due to the substantial resource demands associated with embedding large models. This poses a significant limitation for low-cost or resource-constrained FPGAs, particularly when storing and processing CNN parameters with high memory requirements. To address this issue, existing solutions often rely on high-speed external RAM or exploit the memory subsystems present in SoC platforms combining FPGA and CPU elements. However, these approaches introduce several trade-offs in terms of cost and performance:

- Integrating high-speed external RAM requires the implementation of complex interface protocols on the FPGA, which consumes valuable logic resources. This reduces the available area for the core CNN accelerator, thereby limiting achievable parallelism and degrading overall performance. Additionally, the added hardware cost is non-trivial.
- For optimal performance, data accessed by the accelerator should reside within the FPGA itself to minimize memory latency. Although this can be achieved using fast on-chip memories such as BRAMs, their storage capacity is highly limited. Utilizing high-capacity and high-speed RAM or extensive BRAM blocks significantly increases hardware costs, especially when fully synthesized on-chip.
- Fully connected and densely interconnected layers also introduce challenges. As demonstrated in prior work [37,40], implementing direct inter-layer connections to maximize throughput can consume large amounts of FPGA resources. Even relatively small CNNs may become unsynthesizable on low-end FPGAs under such strategies, requiring more advanced devices.

These challenges highlight the necessity of designing CNN accelerators that judiciously balance parameter storage requirements, inter-layer connectivity, and computational parallelism. To maximize compatibility with FPGAs of varying sizes — including low-end devices — the architecture should enable scalable and modular deployment. This includes support for temporal iteration (reusing computational resources over time) and spatial modularity (partitioning layer functions into parallel components), ensuring flexibility and efficient resource utilization.

3. Architecture of the proposed accelerator

The primary objective of this work is to design and implement a System-on-Chip (SoC) architecture that integrates an ARM-based CPU with a dedicated CNN accelerator, with the aim of improving computational performance and reducing power consumption during the execution of convolutional neural network (CNN) models.

Fig. 1 presents the architecture proposed in this work. It leverages a System-on-Chip (SoC) platform comprising an ARM-based processor coupled with a programmable FPGA fabric to process and analyze signals acquired from an ECG device.

The signal acquisition setup is illustrated in Fig. 2, which displays representative examples of ECG signals obtained from the hardware. This study specifically focuses on two-channel ECG signals acquired via the General Purpose Input/Output (GPIO) interface and the embedded Analog-to-Digital Converter (ADC) of the SoC. The acquisition system samples each channel at a rate of 360 samples per second with 12-bit resolution. The corresponding signal amplitude range is 10 mV, yielding a quantization resolution of 0.8 mV.

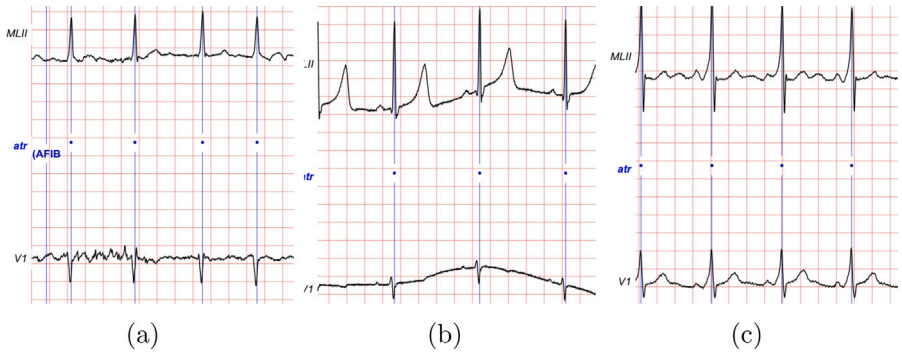


Fig. 2. Examples of input ECG signals - (a) Atrial fibrillation; (b) Pacemaker-regulated activity; (c) Normal sinus rhythm.

As depicted in Fig. 2, the system is capable of capturing various cardiac conditions through the input signals. These include, for example: (a) atrial fibrillation; (b) pacemaker activity; (c) normal cardiac function. These cases are used to evaluate the system's ability to distinguish between different types of heart rhythms based on signal characteristics.

The SoC comprises the following key components: an ARM-based CPU serving as the embedded processing unit, a block RAM (*BRAM*) module used to store CNN model parameters, and custom logic dedicated to executing *Conv2D* (two-dimensional convolutional) and *DENSE* layers. While the implementation primarily targets 2D convolutional layers — given their widespread usage across machine learning applications — the architecture is sufficiently general to support other types of convolutional layers with minor modifications.

A central CNN control unit orchestrates the operations of the convolutional and dense layer accelerators, as well as their interaction with the ARM processor. This controller ensures correct scheduling and communication, enabling the execution of complex CNN models through repeated and coordinated accelerator invocations.

It is important to emphasize that the proposed solution is tailored for resource-constrained, low-power, and cost-effective SoCs. In contrast to existing approaches that deploy complete CNN models on high-capacity FPGAs [37,40], our methodology is guided by the trade-offs inherent in deploying accelerators on more affordable, smaller-scale FPGAs. Consequently, a selective acceleration strategy is adopted, focusing exclusively on convolutional and dense layers, based on the following considerations:

- The available FPGA area is limited, making it impractical to fully implement even moderately sized CNNs in hardware.
- Convolutional and dense layers predominantly involve arithmetic operations that are relatively simple to realize with FPGA resources.
- Other types of layers typically require more complex hardware, making them less suitable for implementation on small-scale FPGAs.
- Convolutional layers exhibit a highly regular computational pattern, characterized by repeated arithmetic operations. This regularity is well-suited to parallel hardware design, allowing iterations to be executed in a single or few clock cycles. Moreover, this repetition enables time-multiplexed access to parallel accelerators in area-constrained designs.
- Dense layers are characterized by a high degree of connectivity among neurons. Efficient hardware acceleration of such layers involves the use of dedicated interconnects. Given the area limitations, a fully parallel implementation is often unfeasible. Instead, a trade-off is achieved by employing iterative interconnect schemes that balance parallelism and latency.
- Neuron parameters, intermediate data, and activations can be stored in FPGA registers for performance benefits, albeit at the cost of increased area. A viable alternative involves storing these data in *BRAM* and leveraging an *ON-CHIP MEMORY* caching strategy to optimize access times without incurring excessive area usage.
- Iterative control of CNN operations can be managed efficiently through custom hardware logic, minimizing overhead and latency.
- Non-accelerated layers can be executed on the CPU. The system's control logic allows the CPU to begin processing as soon as the first iteration of an accelerator completes, thereby improving throughput through pipelined computation.

In the following sections, we describe the architecture of each hardware-accelerated component in detail. Without loss of generality, the focus is placed on two-dimensional convolutional layers, in line with inputs from the datasets used for validation and evaluation. The architecture can be easily adapted to support inputs of different dimensionality; however, such extensions are omitted for brevity.

3.1. Convolutional accelerator

Convolution operations represent the most computationally intensive components in Convolutional Neural Networks (CNNs), accounting for over 90% of total processing workload. Each convolution involves a sliding kernel mechanism combined with a *multiply-and-accumulate* (MAC) operation. In hardware implementations, the convolution between input data and corresponding kernels is realized through three main architectural modules:

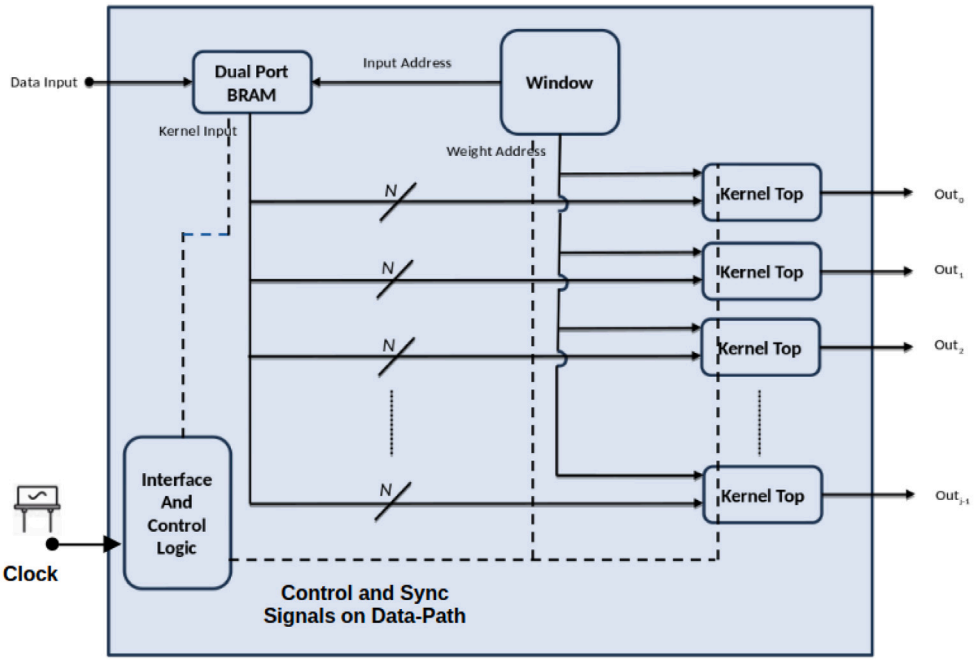


Fig. 3. Hardware architecture of the Conv2D layer accelerator optimized for FPGA implementation. This design integrates key modules to perform efficient 2D convolution on ECG signal data.

- *Dual Port BRAM*
- *Window Memory*
- *Kernel Top*

In addition, here, in Kernel top Module (Fig. 4) ad in Dense Layer (Fig. 6), a module called *Interface and Control Logic* exists. It is the module responsible of control signals orchestration for correct implementation of the Data-Path control in the whole accelerator and it is obviously triggered by a Clock Signal.

Fig. 3 presents the structure of the *Conv2D* layer, which integrates the above-mentioned components. The layer is designed to support $\mathcal{M}$ kernels, each of size $\mathcal{K} \times \mathcal{K}$.

The *Dual Port BRAM* module serves as the primary memory element for storing input data samples. It provides the data required for MAC operations and enables simultaneous read and write operations, facilitating efficient data throughput.

Given the constraints of low-power hardware platforms, it is often infeasible to store the complete set of CNN weights and inputs within on-chip BRAM. To address this limitation, an efficient caching mechanism is introduced to manage the temporary storage of input data and network parameters within BRAM. This mechanism ensures that data can be accessed as needed during computation without exceeding hardware resource limits.

Dense layers require systematic data interconnections between BRAM, intermediate buffer registers and neuron logic. In deep or highly interconnected CNNs, this can result in excessive hardware resource utilization and, in many cases, prevent successful synthesis of the accelerator.

To overcome these limitations, we propose an architecture that facilitates the interconnection between BRAM and accelerator modules through a time-multiplexed approach. This design allows parallel convolution operations to be interleaved temporally, thereby minimizing direct hardware coupling between modules. The key enabling feature of this approach is a *sliding connection window*, which efficiently buffers both weights and input data for use across parallel convolutional accelerators.

To implement the sliding window mechanism, a dedicated *Window Memory* architecture has been developed. This architecture is composed of two core elements: the *Window* module (highlighted in Fig. 3) and the aforementioned *Dual Port BRAM*. The Window module defines a moving window over the input feature map and synchronizes the corresponding convolution operation with the appropriate kernel. It performs dynamic address computation to fetch the relevant input data and associated weights based on kernel dimensions and stride configurations.

The computed input address is forwarded to the Dual Port BRAM module, which retrieves the necessary input sample for processing. Using this address as a reference, the module extracts a column of data values from the input matrix, which are then multiplied and accumulated with the corresponding kernel weights. Concurrently, the weight address is calculated and supplied directly to the *Kernel Top* module, which is responsible for executing the convolution operations.

The *Kernel Top* module, illustrated in Fig. 4, is responsible for executing the convolution operation between the kernel weight matrix and the input data provided by the sliding window. This operation is enabled by the *Window* module, which dynamically

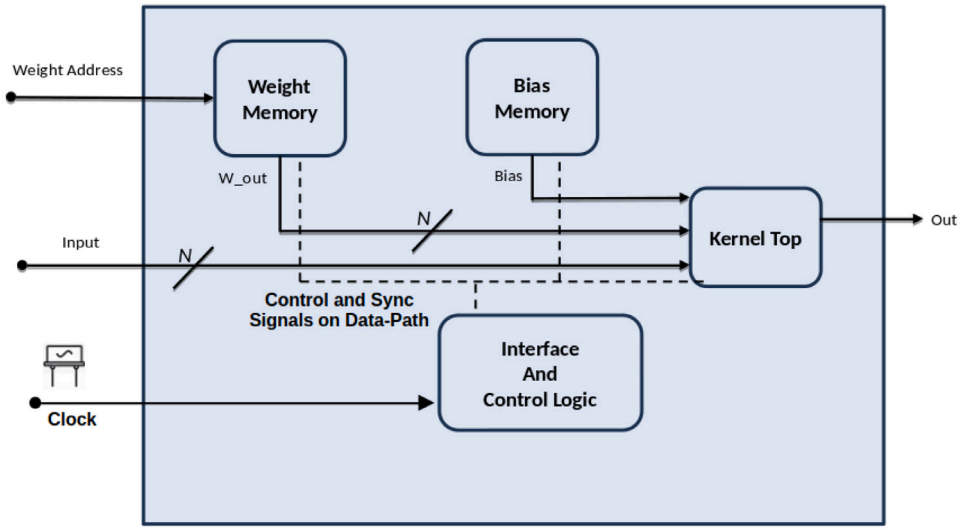


Fig. 4. Structure of the Kernel Top Module responsible for convolution execution. It performs the core multiply-and-accumulate (MAC) operations of the convolution layer by fetching input windows and corresponding kernel weights in parallel. It supports address computation, weight selection, and bias retrieval.

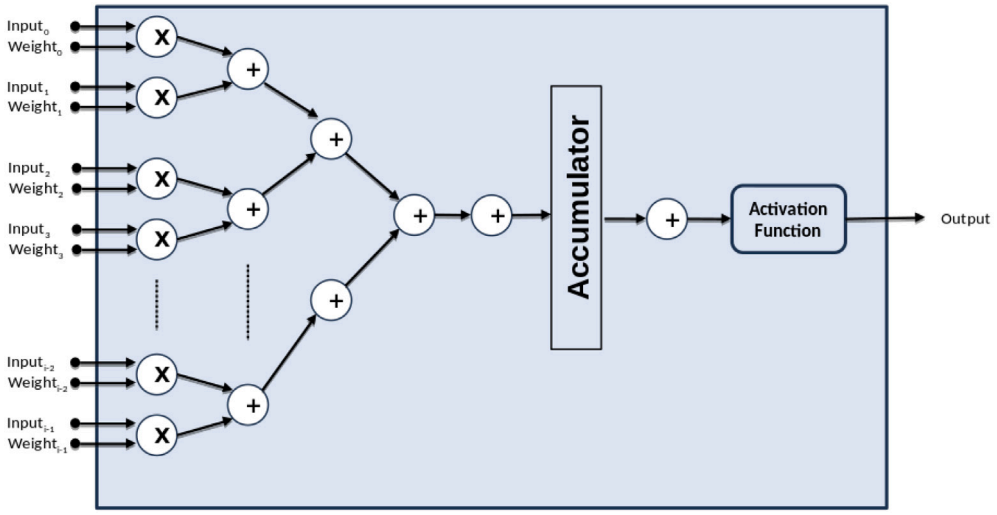


Fig. 5. Detailed architecture of the Kernel Module implementing MAC and activation logic. The module computes the element-wise multiplications between input values and kernel weights using a partially parallel structure optimized for low-cost FPGAs. The products are summed using a tree-based adder.

controls the movement of the connection window across each kernel in the convolutional layer. During each clock cycle, the Window module calculates updated addresses for both the input and kernel weights, allowing the system to fetch the appropriate data required for the multiply-and-accumulate (MAC) operations.

To improve throughput and parallelism, $\mathcal{N}$ read lines are utilized for both input and weight memory, where $\mathcal{N}$ corresponds to the kernel size $\mathcal{K}$. Within the convolutional layer, the Kernel Top module represents an individual filter, maintaining its weights and associated biases in two dedicated memory structures: a *weight memory* and a *bias memory*. The selected weights, input values, and bias are forwarded to the *Kernel* module (see Fig. 5), which performs point-wise multiplication and addition in a fully parallel manner.

A parallel implementation of the convolution process necessitates simultaneous multiplication of input values and corresponding weights, followed by a hierarchical (tree-based) addition of the resulting products. Typically, the convolution kernel is a square matrix with odd dimensions (e.g., 1×1, 3×3, etc.). Odd-sized kernels are favored in practice due to their ability to symmetrically align around the target output pixel. This symmetry simplifies padding strategies and prevents artifacts or misalignments that are often introduced when using even-sized kernels. As a result, even-sized kernels are typically avoided to streamline hardware implementation.

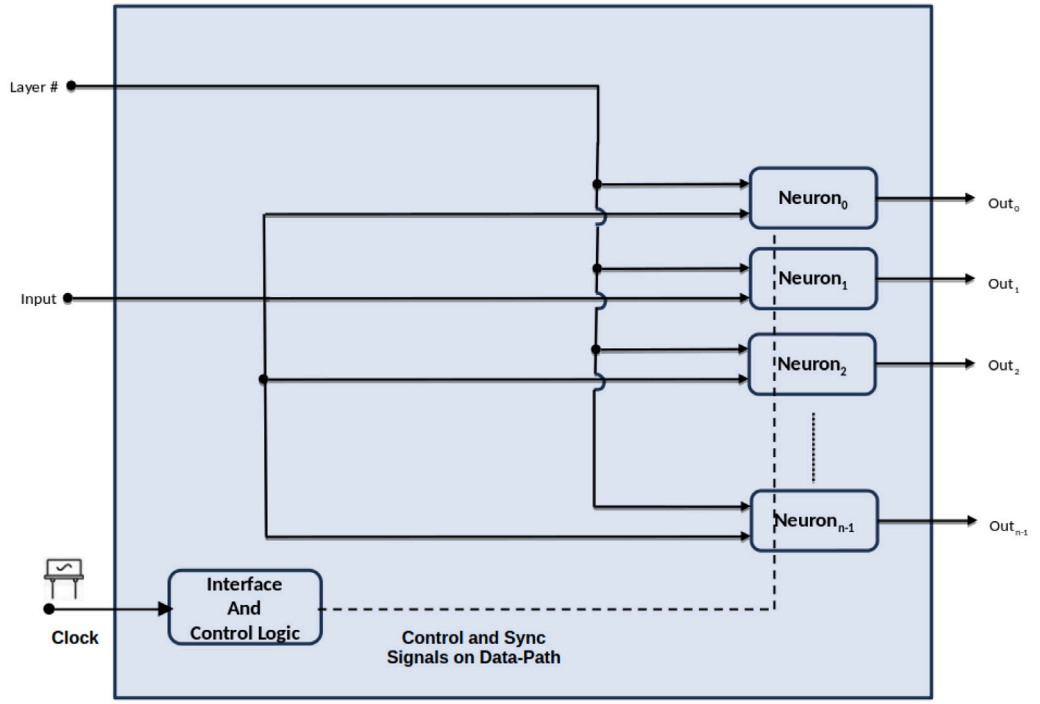


Fig. 6. Hardware implementation of a scalable Dense (Fully Connected) Layer. This architecture supports multiple dense layers and dynamically enables neurons depending on the active layer configuration. Each neuron performs fixed-point MAC operations followed by bias addition and activation.

For a kernel of dimension $K \times K$, the *Kernel* module is designed to use $K-1$ multipliers in conjunction with a tree-based adder structure consisting of $log_2(K-1)$ levels. The final (Kth) multiplier is appended after the last level of the adder tree, and its output is accumulated using an *accumulator* module. Once all partial sums are computed, the result is combined with the corresponding bias and passed through an activation function to generate the final output of the convolutional layer.

The decision to limit the number of multipliers to K instead of using a full $K \times K$ array arises from resource constraints on low-cost FPGAs. In such devices, multipliers are typically implemented using Digital Signal Processing (DSP) blocks, which are available in limited quantity. Consequently, implementing a large number of kernels concurrently may exceed available resources. To address this, a partially sequential implementation is adopted to ensure compatibility with hardware constraints while still maintaining efficient performance. A fully sequential approach, while feasible, would underutilize the inherent parallelism of FPGAs and reduce the overall system performance.

3.2. Dense layer

According to the general structure of dense layers, the implementation of a fully connected (dense) layer on an FPGA is illustrated in Fig. 6. This architecture includes the necessary interface and control logic to manage the initiation of operations and to enable communication with other system components. Each dense layer consists of multiple computational units referred to as *neurons*. The proposed design is both modular and reusable, supporting the integration of up to $\mathcal{M}$ dense layers within the neural network architecture. This scalability facilitates the deployment of complex models while maintaining compatibility with resource-constrained FPGA platforms.

For a convolutional neural network with $\mathcal{M}$ consecutive dense layers, the number of neurons in the module is determined as follows:

$$n = max(D), \quad D = \{d_j\}_{j=1}^{M} \tag{1}$$

where:

- n is the maximum number of neurons required
- D is an array containing the number of neurons of the $\mathcal{M}$ dense layers of the neural network

Depending on the currently active dense layer, a certain number of neurons are enabled or disabled.

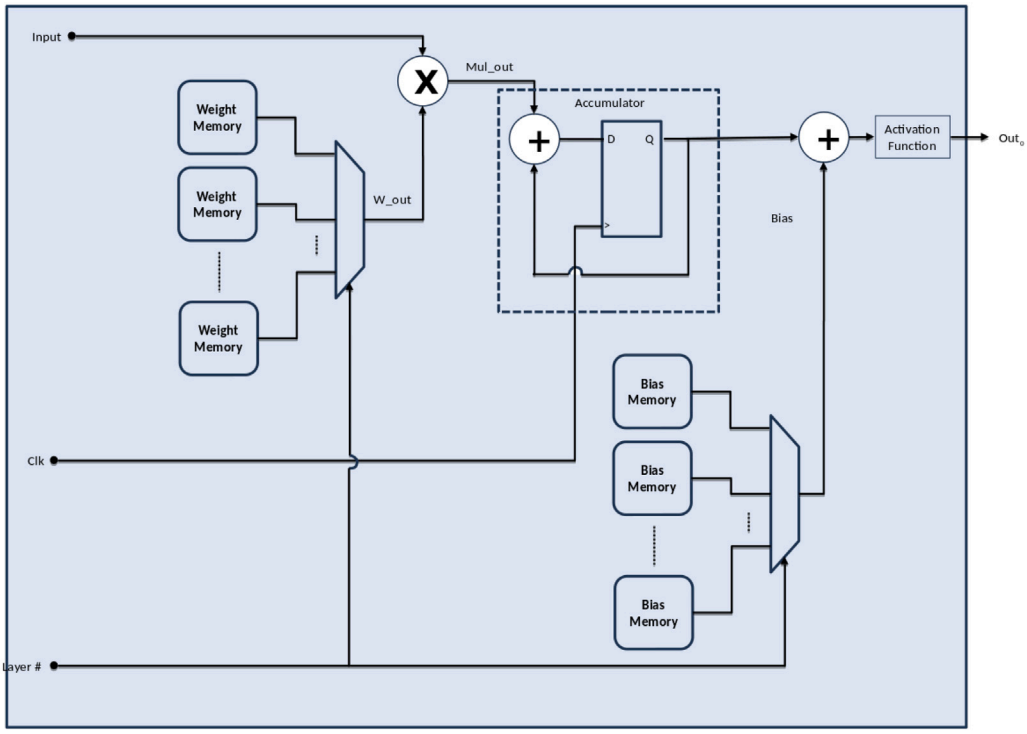


Fig. 7. Internal architecture of a hardware-implemented Neuron used in dense layers. Each neuron performs fixed-point multiply-and-accumulate (MAC) operations by reading input activations and weights from on-chip memory. The design supports efficient reuse across multiple dense layers through a layer selection signal (layer #), which dynamically controls memory multiplexing.

3.3. Neuron architecture

The hardware implementation of the *Neuron Architecture*, derived from the canonical structure of an artificial neuron, is presented in Fig. 7. In this design, each neuron retrieves input values from the on-chip memory and performs element-wise multiplications with the corresponding weights. The resulting products are accumulated through a dedicated accumulator unit to complete the multiply-accumulate (MAC) operations. Once all inputs have been processed, the accumulated sum is combined with a bias term and subsequently passed through a configurable activation function. The final output of the neuron is then written back to the on-chip memory for further processing.

The signal *layer #* contains the information required to select the appropriate active dense layer to be used in the Neuron Module. This signal enables the multiplexing of the corresponding Weight and Bias memories, ensuring the correct operation for the desired layer. It originates from the *Interface and Control Unit* module, previously introduced. This mechanism allows for the reuse of Neuron Modules to compute neuron outputs across different layers. Additionally, multiplexing the Weight and Bias data reduces the number of connection lines when managing multiple neurons. These parameters are accessed through the previously described windowed BRAM mechanism, which ensures efficient data retrieval and reuse across multiple layers.

By multiplexing both memory space and operations over time, the design reduces the number of required neuron accelerators and connection lines, enabling synthesis on resource-constrained FPGAs.

4. Experimental setup

The evaluation of the proposed architecture is conducted using the *MIT-BIH Arrhythmia Database* [41]. This dataset comprises 48 half-hour segments of two-channel ambulatory ECG recordings collected from 47 subjects. To enhance the representation of clinically significant but infrequent arrhythmias, an additional 25 recordings were incorporated from the same population cohort. These less common arrhythmias were underrepresented in the initial random selection but were found in a broader dataset of 4000 24-h ambulatory ECG recordings. These recordings were obtained from a demographically diverse patient population, consisting of approximately 60% inpatients and 40% outpatients at Beth Israel Hospital in Boston.

Of the supplementary recordings, 23 were selected at random, while the remaining 2 were chosen to ensure adequate representation of rare arrhythmia types. The final dataset includes five distinct heartbeat classes, defined as follows:

- N : non-ecotic beats (normal beat);

Table 1
Training and test sets original size.

Type	# sample
Training set	87,554
Test set	21,892
Total samples	**109,446**

Table 2
Class distribution before and after resampling. The table reports the percentage of samples per class — Normal and Supraventricular (N&S), Ventricular (V), Fusion (F), Unknown (Q) — illustrating the significant class imbalance in the original dataset and the uniform distribution achieved through resampling.

	N	S	V	F	Q
Pre-Resampling	82.8%	2.5%	6,6%	0.7%	7.3%
Post-Resampling	20%	20%	20%	20%	20%

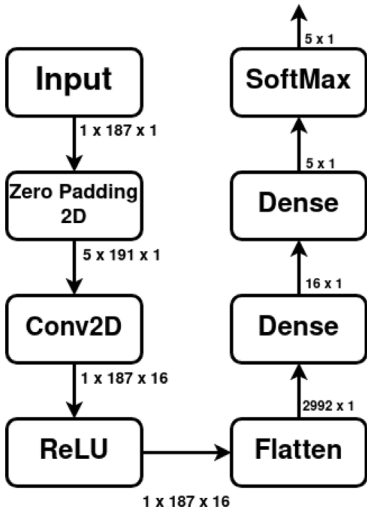


Fig. 8. Structure of CNN model.

- S : supraventricular ectopic beats;
- V : ventricular ectopic beats;
- F : fusion beats;
- Q: unknown beats;

The original dataset contains almost 110.000 samples which are pre-divided into *training* and *test* subsets, as summarized in Table 1

The original distribution of samples is imbalanced across the five heartbeat classes previously described. Models trained on such imbalanced datasets often exhibit degraded generalization performance, particularly when predicting underrepresented classes. To mitigate this issue and improve classification accuracy, we balanced the training dataset using a resampling technique provided by the *scikit-learn* library.[1] Specifically, we applied over-sampling to increase the number of training samples to 50,000 per class, resulting in a total of 250,000 samples—up from the original 87,554. The class distributions before and after resampling are summarized in Table 2.

Fig. 8 illustrates the architecture of the convolutional neural network (CNN) employed for arrhythmia detection from electrocardiogram (ECG) signals. In contrast to many existing approaches in the literature, this network is a custom implementation obtained from a publicly available GitHub repository,[2] which significantly facilitated the hardware design and deployment process.

An electrocardiogram (ECG) is a non-invasive diagnostic test commonly used to evaluate the heart's rhythm and electrical activity. The network accepts an input signal of size $(1 \times 187 \times 1)$, which undergoes an initial padding operation, producing an intermediate tensor of size $(5 \times 191 \times 1)$. A 2D convolution is then applied using 16 kernels of size (5×5), resulting in an output of size $(1 \times 187 \times 16)$. This output is subsequently flattened and passed through two fully connected (dense) layers containing 16 and

[1] https://scikit-learn.org/stable/modules/generated/sklearn.utils.resample.html.
[2] https://github.com/dhruvchikhaliya12/ECG_tinyML.

Table 3
Main features of baseline and Quantized models.

Model	Data-Type	bit-width	Accuracy
Baseline Model	Floating point	32	95.0%
Quantized Model	Fixed point	7	91.9%

Table 4
Resource usage.

Resource type	Used	Available	% Use
LUT	20 198	53 200	37.70
LUTRAM	1439	17 400	8.27
FF	12 933	106 400	12.16
BRAM	50.50	140	36.07
DSP	98	220	44.55
BUFG	1	32	3.13

5 neurons, respectively. The final dense layer's output is processed by a softmax activation function to compute class probabilities, where each output neuron corresponds to one of the target heartbeat classes. To evaluate the hardware-accelerated architecture, we trained two versions of the model described above: one designed for execution on a standard CPU, serving as a baseline, and another tailored for deployment on the proposed accelerator.

The second model is quantized using the QKeras library,[3] which allows for the substitution of standard layers with their quantized counterparts and enables specification of fixed-point bit widths. In this implementation, all layers — including activation functions — are quantized to 7-bit fixed-point representations, with the exception of the final softmax layer, which remains in floating-point format to preserve classification accuracy.

Table 3 resumes the main features of the two models.

5. Results

This section presents the results obtained from the experimental setup described in Section 4. All results refer specifically to the *Quantized Model* introduced in the previous section.

The performance of the proposed architecture was evaluated on the Zybo Z7-20 development board, using the following primary metrics:

- Power Consumption
- Resource Utilization
- Response Time

Furthermore, to compare the CPU-based and FPGA-based implementations, the following evaluation criteria were considered: *inference time*, *hardware cost*, and the *cost-performance trade-off* associated with the hardware deployment.

To minimize energy consumption, the system clock on the Zybo Z7-20 board was configured to operate at *50* MHz. While the architecture is capable of supporting clock frequencies exceeding *260* MHz, the exploration of maximum performance boundaries falls outside the scope of this study.

Resource Utilization. Following the synthesis and implementation stages performed using Xilinx Vivado, the FPGA resource usage was analyzed. The summary of available and utilized hardware resources is presented in Table 4.

The most resource-constrained components in the proposed implementation are the Digital Signal Processors (DSPs). In the synthesis process, Vivado maps each multiplication operation to a DSP block. Since neural networks inherently rely heavily on multiplication operations, this constraint significantly limits the level of parallelism achievable on the FPGA. Once the available DSPs are fully utilized, Vivado resorts to using Look-Up Tables (LUTs) to synthesize additional multipliers. On average, each excess multiplier synthesized using LUTs consumes approximately 1500 LUTs, which can quickly deplete other critical FPGA resources. Therefore, it is essential to remain within the DSP budget to avoid compromising the overall architecture due to excessive LUT usage.

In our implementation, the convolutional layer employs 16 parallel kernels, each configured with 5 parallel multipliers to perform convolution operations efficiently. For the dense layer, 16 neurons are implemented in parallel, each utilizing a single multiplier.

The total number of DSPs used in the architecture is calculated as follows:

$$DSP_{\text{tot}} = DSP_{\text{conv2D}} + DSP_{\text{dense}} + DSP_{\text{other}} = (16 \cdot 5) + (16 \cdot 1) + 2 = 98$$

The term DSP_{other} accounts for two additional multipliers used within the Window Memory module, which Vivado also synthesized as DSP blocks.

[3] https://github.com/google/qkeras.

Table 5
Power consumption.

State	Power (Wh)	Current (mA)
Turned On	0.45	92
Execution	0.60	120

Given the constraints of the target hardware, further increases in parallelism are infeasible. For example, implementing 25 multipliers per kernel — equivalent to the kernel size — would yield full parallelization. However, this would result in a requirement of 400 DSPs (computed as 16 kernels × 25 multipliers per kernel), which significantly exceeds the number of available DSPs on the Zybo Z7-20 FPGA board.

Power Consumption. To more accurately isolate the power consumed by the FPGA component itself (excluding the contribution from the entire development board), measurements were taken across three distinct operational states:

- **Turned On**: The board is powered on, but no firmware is loaded. Only the power LED is active.
- **Execution**: The neural network firmware is loaded and actively running. Both the power LED and the firmware execution LED are illuminated.
- **After Execution**: The board has completed execution and is in an idle state, with only the processor active. Both LEDs remain illuminated.

Table 5 summarizes the measured power consumption values. Each measurement was averaged over a continuous 20 min period under a stress-test scenario, where the board was supplied with input samples without interruption throughout the entire duration. Power consumption was found to be consistent across runs, and no noticeable difference was observed between executions with or without the final Softmax layer, indicating that its impact on overall consumption is negligible.

The two operational states of principal interest for assessing the FPGA's power consumption are the *Turned On* and *Execution* phases. During the *Execution* phase, the FPGA completes its processing tasks and remains idle, with only the embedded processor continuing to draw power. By computing the difference in power usage between the *Turned On* and *Execution* phases, it was determined that the FPGA core alone consumes approximately *0.15 Wh* and *28 mAh*.

Performance and Cost Comparison: CPU vs FPGA — This section presents a comparative analysis of the accelerator's performance against two CPU-based implementations: (i) a high-end CPU, and (ii) the ARM Cortex-A9 processor embedded on the Zybo Z7-20 board (operating without hardware acceleration). All measurements were conducted under a stress-test scenario, in which all systems continuously processed input samples.

It is important to note that the high-end CPU incurs a significantly higher cost compared to a single FPGA unit, which underscores the cost-effectiveness of the FPGA-based solution. Furthermore, inference time analysis revealed that the FPGA implementation consistently outperformed the CPU. Specifically, inference times were recorded over a range of input sizes, from 1 up to 1,500,000 samples. Across all test cases, the FPGA exhibited superior performance, achieving up to 300 × speedup in average inference time. For individual samples, the FPGA achieved a speedup of approximately 2000 × compared to the CPU.

When comparing the performance of the ARM-based system with and without the accelerator, it was observed that direct experimental measurement for sample sizes of 600,000 and 1,500,000 was impractical due to excessively long runtimes. Consequently, the performance data for these cases were estimated based on extrapolation from smaller datasets.

Fig. 9 resumes performances comparisons between the systems without and with accelerator.

It is also noteworthy that further evaluations were performed with and without the inclusion of the *Softmax* stage, which is executed in software. The Softmax function constitutes the slowest segment of the processing pipeline and thus introduces a bottleneck, limiting the achievable throughput. This reinforces the advantage of excluding or offloading the Softmax stage when optimizing for performance.

The inference time exhibits a clear dependence on the presence of the *Softmax* stage. Specifically, as the number of samples to be classified increases, the implementation *excluding* the Softmax stage consistently achieves lower inference times compared to the implementation *including* it. While the performance gap is relatively small for a limited number of samples, it becomes significantly more pronounced at larger scales, indicating that the computational overhead introduced by the Softmax function grows with the input size (Table 6).

The reference processing unit for this analysis is the *dual-core ARM Cortex-A9* integrated into the Zybo Z7-20 board.

Overall, the FPGA-based implementation demonstrates superior performance compared to the CPU-based solution in terms of both inference time and hardware cost. Furthermore, the FPGA implementation offers significantly reduced power consumption, which is a crucial consideration for Internet of Things (IoT) applications. It is important to emphasize that these performance advantages are contingent upon several factors, including the specific characteristics of the target problem, the architectural configuration of the neural network, and the format and properties of the input data.

5.1. State of the art comparison

Table 7 presents a comparative overview of several CNN hardware accelerators implemented on FPGA platforms. For readability, the results are split across two tables; however, the main objective is to evaluate the effectiveness of our proposed solution against

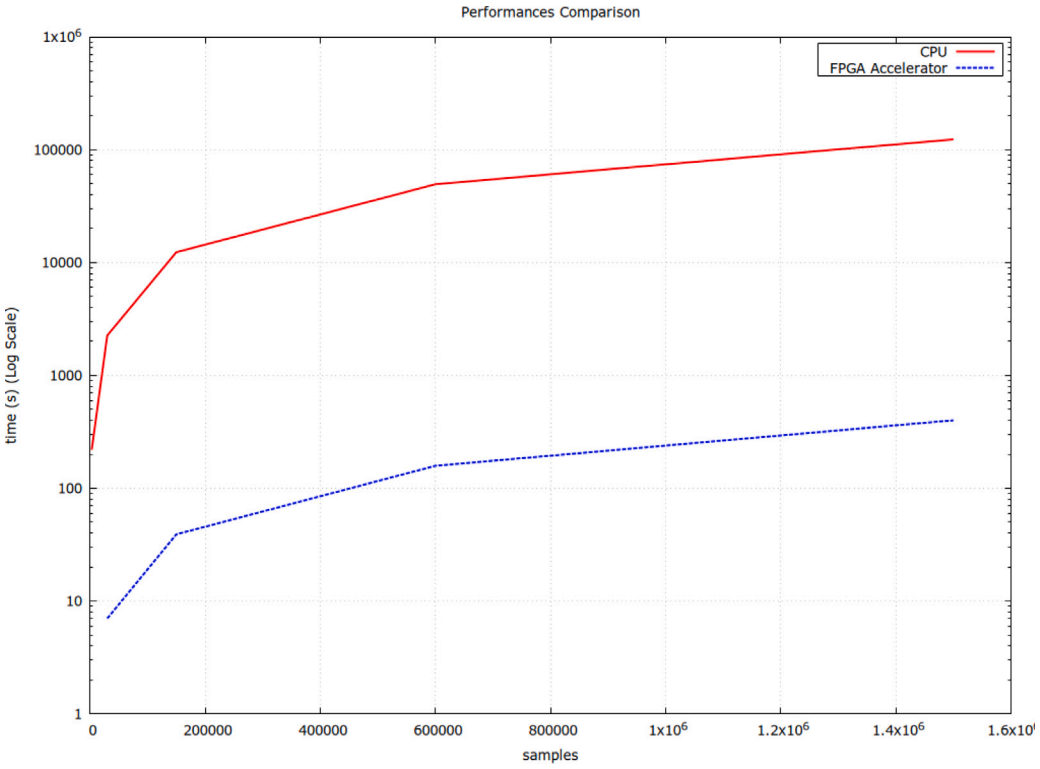


Fig. 9. Performances comparison.

Table 6
Comparison of inference times (in milliseconds) between CPU and FPGA implementations for varying numbers of samples. Results are shown for FPGA with and without the Softmax stage.

Samples	CPU	FPGA(with Softmax)	FPGA (no Softmax)
1	0.627	0.000262	0.000259
3000	219.6	0.786	0.779
30 000	2253	7.91	7.87
150 000	12 343	39.85	39.07
600 000	49 372[a]	158.34	156.46
1 500 000	123 430[b]	400.01	394.54

[a] Estimated value.
[b] Estimated value.

these existing approaches using a consistent set of metrics: FPGA device, frequency, numerical precision, computational throughput (GOP/s), and power efficiency (GOP/s/W).

While most solutions target well-established CNN architectures such as VGG16 and AlexNet and are implemented on high-end FPGA platforms (e.g., XC7Z045, Virtex-7, and Arria10), they typically achieve higher throughput, with the peak value of 445.6 GOP/s reported in [42]. However, their power efficiency remains moderate, with values ranging from 5.80 to 44.48 GOP/s/W. These results indicate a trade-off between raw computational power and energy efficiency, with designs often favoring the former.

More recent and efficient designs also target VGG-based models using 8-bit quantization on Zynq UltraScale+ devices. Although these implementations achieve very high performance (e.g., 990.8 GOP/s in [43]), their power efficiencies, while improved, still range below 65 GOP/s/W.

In contrast, our proposed design, although achieving a modest throughput of 50.2 GOP/s, demonstrates exceptional power efficiency of 83.67 GOP/s/W, the highest among all compared works. This outcome is achieved despite the use of a more constrained and cost-effective FPGA platform (Zybo Z7-20) operating at a significantly lower frequency (50 MHz) and employing 7-bit fixed-point quantization. The result highlights the suitability of our solution for low-power, edge-oriented applications where energy efficiency is a critical constraint.

Table 7
Comparison of CNN Accelerator Implementations on FPGA Platforms.

Reference	CNN Type	Device (FPGA)	Precision	Freq. (MHz)	Perf. (GOP/s)	Power Eff. (GOP/s/W)
[44]	VGG16	Zynq XC7Z045	16-bit fp	150	136.97	14.22
[45]	nn-X	Zynq XC7Z045	16-bit fp	142	23.18	5.80
[42]	AlexNet	Virtex7 VX485T	16-bit fp	100	445.6	17.97
[46]	SparkNet	Arria10 GX1150	16-bit fp	150	337.2	44.48
[47]	VGG16	Zynq ZC706	8-bit fp	200	316.4	16.14
[47]	ResNet50	Zynq ZC706	8-bit fp	200	328.2	16.74
[43]	VGG16	Zynq ZCU102	8-bit fp	200	990.8	57.94
[43]	AlexNet	Zynq ZCU102	8-bit fp	200	476.7	30.95
[48]	VGG16	Zynq XC7Z045	8-bit fp	150	112.8	8.78
[49]	VGG16	Zynq Ultra96v2	8-bit int	333	153.5	30.10
[50]	VGG16	Zynq ZCU102	8-bit fp	200	393.2	64.45
[50]	AlexNet	Zynq ZCU102	8-bit fp	200	280.3	45.95
This Work	Ad-Hoc	Zybo Z7–20	7-bit fix	50	50.2	**83.67**

In summary, while other designs excel in raw performance on more capable hardware, our implementation offers a highly efficient alternative that maximizes performance-per-watt, making it particularly advantageous for embedded and power-sensitive deployment scenarios.

6. Threats to validity

We outline here potential threats to the validity of our results concerning the design and evaluation of the proposed FPGA-based architecture for arrhythmia detection using Convolutional Neural Networks (CNNs). The *Internal Validity* of results is primarily influenced by the characteristics of the training dataset and the quantization strategy applied to the CNN model. While the MIT-BIH Arrhythmia Database is a widely recognized benchmark, preprocessing steps such as resampling and class balancing may introduce biases. Moreover, the fixed-point quantization to 7-bit representation, while essential for hardware efficiency, may impact the fidelity of model predictions, especially for borderline or noisy ECG signals.

For what *External Validity* concerns, the generalizability of our findings to other ECG datasets or real-time clinical scenarios is limited. The experimental setup assumes input data acquisition and preprocessing under controlled conditions. In practical applications, variations in signal quality, sensor placement, and patient-specific physiology may reduce the classification accuracy. Additionally, while our system is demonstrated on the Zybo Z7-20 board, hardware-specific optimizations may not directly translate to different FPGA platforms or IoT nodes.

In addition, we have to take into account that our evaluation metrics focus primarily on classification accuracy, power efficiency (GOP/s/W), and inference latency. Although these indicators are relevant for assessing the hardware performance, they do not directly measure clinical effectiveness or explainability, which are crucial in medical device certification and adoption. Furthermore, the exclusion of the Softmax layer from hardware acceleration introduces a software dependency that may influence total system latency.

Finally, due to resource constraints, performance measurements such as inference time and energy consumption were averaged over a limited set of benchmarks and hardware configurations. The extrapolation used for large batch sizes on ARM-only systems may not capture all real-world variations. In addition, synthesis and implementation were conducted using a single toolchain (Xilinx Vivado), which may yield different results under other synthesis environments or compiler optimizations.

7. Conclusions

This work introduces a novel FPGA-based implementation of a Convolutional Neural Network (CNN) for arrhythmia detection using electrocardiogram (ECG) signals. The proposed solution demonstrates that high-speed CNN inference can be achieved even on cost-effective, low-power FPGA platforms. The architecture is fully scalable and adaptable across various FPGA devices, offering flexibility in selecting the appropriate trade-off among hardware complexity, power consumption, and computational performance.

A key distinguishing aspect of the design lies in the efficient mapping of CNN computational components onto the FPGA fabric, along with the optimized management of memory resources used to store CNN parameters, inputs, and intermediate feature maps. The system has been validated on the Zybo Z7-20 development board, leveraging the MIT-BIH Arrhythmia Database for a continuous processing duration of 20 min.

In the proposed CNN implementation, the convolutional layer comprises 16 parallel kernels, each employing five parallel multipliers to execute convolution operations. The dense (fully connected) layer includes 16 neurons operating in parallel, each using a single multiplier. Experimental results yielded the following key outcomes: (a) power consumption of approximately 0.15 Wh, (b) average FPGA resource utilization below 25%, and (c) low response latency.

Moreover, comparative analysis against a software-only execution on a dual-core ARM Cortex-A9 CPU highlights the efficiency of the hardware implementation. The FPGA-based accelerator achieved over a 300 × speedup in inference performance on average, with peaks reaching up to 2000× for single-sample evaluations.

CRediT authorship contribution statement

Luca Greco: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Francesco Moscato:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Pierluigi Ritrovato:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Mario Vento:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Luca Greco reports financial support was provided by Italian Ministry of University and Research (MUR). If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We acknowledge financial support under the National Recovery and Resilience Plan (NRRP), Italy, Mission 4, Component 2, Investment 1.1, Call for tender No. 1409 published on 14.9.2022 by the Italian Ministry of University and Research (MUR), funded by the European Union–NextGenerationEU–Project "Entrust: usEr ceNtric plaTform foR continoUS healThcare"–CUP:D53D23017360001 - Grant Assignment Decree No. 1382 adopted on 01-09-2023 by the Italian MUR.

Data availability

Data will be made available on request.

References

[1] Fatos Xhafa, Machine Learning, Big Data, and IoT for Medical Informatics, Academic Press, 2021.
[2] Agbotiname Lucky Imoize, Mohammad S Obaidat, Houbing Herbert Song, Fatos Xhafa, Federated learning for digital healthcare systems, 2024.
[3] Fernanda Famá, José N. Faria, David Portugal, An IoT-based interoperable architecture for wireless biomonitoring of patients with sensor patches, Internet Things 19 (2022) 100547.
[4] Trung Pham Dinh, Cuong Pham-Quoc, Tran Ngoc Thinh, Binh Kieu Do Nguyen, Pham Cong Kha, A flexible and efficient FPGA-based random forest architecture for IoT applications, Internet Things 22 (2023) 100813.
[5] Mhd Rashed Al Koutayni, Gerd Reis, Didier Stricker, DeepEdgeSoC: End-to-end deep learning framework for edge IoT devices, Internet Things 21 (2023) 100665.
[6] Zhancheng Zhang, Xiaoqing Luo, Heartbeat classification using decision level fusion, Biomed. Eng. Lett. 4 (2014) 388–395.
[7] Shu Lih Oh, Eddie Y.K. Ng, Ru San Tan, U. Rajendra Acharya, Automated diagnosis of arrhythmia using combination of CNN and lstm techniques with variable length heart beats, Comput. Biol. Med. 102 (2018) 278–287.
[8] Zhenyu Zheng, Zhencheng Chen, Fangrong Hu, Jianming Zhu, Qunfeng Tang, Yongbo Liang, An automatic diagnosis of arrhythmias using a combination of CNN and LSTM technology, Electron. 9 (1) (2020) 121.
[9] Viraj Rawal, Priyank Prajapati, Anand Darji, Hardware implementation of 1D-CNN architecture for ECG arrhythmia classification, Biomed. Signal Process. Control. 85 (2023) 104865.
[10] VB.K.L. Aruna, E. Chitra, M. Padmaja, Accelerating deep convolutional neural network on FPGA for ECG signal classification, Microprocess. Microsyst. 103 (2023) 104939.
[11] Chao Chen, Bruno da Silva, Chenxi Yang, Caiyun Ma, Jianqing Li, Chengyu Liu, Automlp: A framework for the acceleration of multi-layer perceptron models on FPGAs for real-time atrial fibrillation disease detection, IEEE Trans. Biomed. Circuits Syst. (2023).
[12] Kayode Inadagbo, Baran Arig, Nisanur Alici, Murat Isik, Exploiting FPGA capabilities for accelerated biomedical computing, in: 2023 Signal Processing: Algorithms, Architectures, Arrangements, and Applications, SPA, IEEE, 2023, pp. 48–53.
[13] Hengchang Hu, Bo Liu, Pan Zhang, Several models and applications for deep learning, in: 2017 3rd IEEE International Conference on Computer and Communications, ICCC, IEEE, 2017, pp. 524–530.
[14] Ce Zhang, Xin Pan, Huapeng Li, Andy Gardiner, Isabel Sargent, Jonathon Hare, Peter M Atkinson, A hybrid MLP-cnn classifier for very fine resolution remotely sensed image classification, ISPRS J. Photogramm. Remote Sens. 140 (2018) 133–144.
[15] Mengmeng Zhang, Wei Li, Qian Du, Diverse region-based CNN for hyperspectral image classification, IEEE Trans. Image Process. 27 (6) (2018) 2623–2634.
[16] U Rajendra Acharya, Hamido Fujita, Oh Shu Lih, Yuki Hagiwara, Jen Hong Tan, Muhammad Adam, Automated detection of arrhythmias using different intervals of tachycardia ECG segments with convolutional neural network, Inform. Sci. 405 (2017) 81–90.
[17] Hongbo Gao, Bo Cheng, Jianqiang Wang, Keqiang Li, Jianhui Zhao, Deyi Li, Object classification using CNN-based fusion of vision and LIDAR in autonomous vehicle environment, IEEE Trans. Ind. Informatics 14 (9) (2018) 4224–4231.
[18] S. Yuan, et al., ECG arrhythmia classification using depthwise separable convolutions on FPGA, IEEE Trans. Biomed. Circuits Syst. 15 (2) (2021) 434–443.
[19] J. Zhang, et al., Accelerating CNN-based ECG classification on FPGA using optimal memory management, in: Proceedings of the International Conference on Field-Programmable Technology, FPT, 2019.
[20] X. Cheng, et al., Automatic design space exploration for FPGA-based convolutional neural network implementations, in: Proceedings of the International Symposium on Field-Programmable Gate Arrays, FPGA, 2020.

[21] A.Y. Ma, et al., Pruning-based FPGA implementation of ECG arrhythmia classification networks, IEEE Trans. Circuits Syst. I. Regul. Pap. 68 (6) (2021) 2403–2414.
[22] K. Patel, et al., Systolic array-based FPGA implementation of convolutional neural networks for ECG classification, IEEE Trans. Biomed. Eng. 67 (2) (2020) 565–575.
[23] R. Parizi, et al., Efficient ECG classification on FPGAs using customized accelerator: Toward low-power real-time monitoring systems, IEEE Trans. Circuits Syst. II: Express Briefs 68 (10) (2021) 3834–3838.
[24] S. Mao, et al., FPGA accelerated ECG arrhythmia classification, in: IEEE International Symposium on Circuits and Systems, ISCAS, 2019.
[25] Aneta Poniszewska-Maranda, Daniel Kaczmarek, Natalia Kryvinska, Fatos Xhafa, Studying usability of AI in the IoT systems/paradigm through embedding NN techniques into mobile smart service system, Computing 101 (11) (2019) 1661–1685.
[26] Mohsine Eleuldj, et al., Survey of deep learning neural networks implementation on FPGAs, in: 2020 5th International Conference on Cloud Computing and Artificial Intelligence: Technologies and Applications (CloudTech), IEEE, 2020, pp. 1–8.
[27] Tejalal Choudhary, Vipul Mishra, Anurag Goswami, Jagannathan Sarangapani, A comprehensive survey on model compression and acceleration, Artif. Intell. Rev. 53 (7) (2020) 5113–5155.
[28] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, Xiaotong Zhang, Pruning and quantization for deep neural network acceleration: A survey, Neurocomputing 461 (2021) 370–403.
[29] Song Han, Jeff Pool, John Tran, William Dally, Learning both weights and connections for efficient neural network, Adv. Neural Inf. Process. Syst. 28 (2015).
[30] Suraj Srinivas, R. Venkatesh Babu, Data-free parameter pruning for deep neural networks, 2015, arXiv preprint arXiv:1507.06149.
[31] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, Hans Peter Graf, Pruning filters for efficient convnets (2016), 2017, arXiv preprint arXiv:1608.08710.
[32] Venkata Anuhya Ardeti, Venkata Ratnam Kolluru, George Tom Varghese, Rajesh Kumar Patjoshi, An overview on state-of-the-art electrocardiogram signal processing methods: Traditional to AI-based approaches, Expert Syst. Appl. 217 (2023) 119561.
[33] G.B. Moody, R.G. Mark, The impact of the MIT-bih arrhythmia database, IEEE Eng. Med. Biol. Mag. 20 (3) (2001) 45–50.
[34] Jinhai Hu, Wang Ling Goh, Yuan Gao, Classification of ECG anomaly with dynamically-biased LSTM for continuous cardiac monitoring, in: 2023 IEEE International Symposium on Circuits and Systems, ISCAS, IEEE, 2023, pp. 1–5.
[35] Ming-Yueh Ku, Tai-Siang Zhong, Yi-Ting Hsieh, Shuenn-Yuh Lee, Ju-Yi Chen, A high performance accelerating CNN inference on FPGA with arrhythmia classification, in: 2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems, AICAS, IEEE, 2023, pp. 1–4.
[36] Chen Zhang, Jian Li, Pengfei Guo, Qiuping Li, Xing Zhang, et al., A configurable hardware-efficient ECG classification inference engine based on CNN for mobile healthcare applications, Microelectron. J. 141 (2023) 105969.
[37] David Gschwend, Zynqnet: An fpga-accelerated embedded convolutional neural network, 2020, arXiv preprint arXiv:2005.06892.
[38] Zhichen Wang, Hengyi Li, Xuebin Yue, Lin Meng, Briefly analysis about CNN accelerator based on FPGA, Procedia Comput. Sci. 202 (2022) 277–282.
[39] Yonghua Zhang, Haojie Wang, Zhenhua Pan, An efficient CNN accelerator for pattern-compressed sparse neural networks on FPGA, Neurocomputing 611 (2025) 128700.
[40] Xin Zhou, Wei Xie, Han Zhou, Yongjing Cheng, Ximing Wang, Yun Ren, Shandong Yuan, Liuwen Li, An accelerated FPGA-based parallel CNN-LSTM computing device, IEEE Access (2024).
[41] George B. Moody, Roger G. Mark, The impact of the MIT-bih arrhythmia database, IEEE Eng. Med. Biol. Mag. 20 (3) (2001) 45–50.
[42] Zhiqiang Liu, Yong Dou, Jingfei Jiang, Jinwei Xu, Shijie Li, Yongmei Zhou, Yingnan Xu, Throughput-optimized FPGA accelerator for deep convolutional neural networks, ACM Trans. Reconfigurable Technol. Syst. (TRETS) 10 (3) (2017) 1–23.
[43] Chaoyang Zhu, Kejie Huang, Shuyuan Yang, Ziqi Zhu, Hejia Zhang, Haibin Shen, An efficient hardware accelerator for structured sparse convolutional neural networks on FPGAs, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 28 (9) (2020) 1953–1965.
[44] Jiantao Qiu, Jie Wang, Song Yao, Kaiyuan Guo, Boxun Li, Erjin Zhou, Jincheng Yu, Tianqi Tang, Ningyi Xu, Sen Song, et al., Going deeper with embedded FPGA platform for convolutional neural network, in: Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, 2016, pp. 26–35.
[45] Vinayak Gokhale, Jonghoon Jin, Aysegul Dundar, Berin Martini, Eugenio Culurciello, A 240 g-ops/s mobile coprocessor for deep neural networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops, 2014, pp. 682–687.
[46] Ming Xia, Zunkai Huang, Li Tian, Hui Wang, Victor Chang, Yongxin Zhu, Songlin Feng, SparkNoC: An energy-efficiency FPGA-based accelerator using optimized lightweight CNN for edge computing, J. Syst. Archit. 115 (2021) 101991.
[47] Ning Li, A high energy-efficiency inference accelerator exploiting sparse CNNs, in: Proceedings of the 2020 6th International Conference on Computing and Artificial Intelligence, 2020, pp. 534–541.
[48] Weijie You, Chang Wu, RSNN: A software/hardware co-optimized framework for sparse convolutional neural networks on FPGAs, IEEE Access 9 (2020) 949–960.
[49] Tianyang Yu, Bi Wu, Ke Chen, Chenggang Yan, Weiqiang Liu, Data stream oriented fine-grained sparse CNN accelerator with efficient unstructured pruning strategy, in: Proceedings of the Great Lakes Symposium on VLSI 2022, 2022, pp. 243–248.
[50] Yonghua Zhang, Haojie Wang, Zhenhua Pan, An efficient CNN accelerator for pattern-compressed sparse neural networks on FPGA, Neurocomputing 611 (2025) 128700.